

Product Requirements Document: Rate Limiting API Service

Version: 1.0

Last Updated: February 2, 2026

Document Owner: Engineering Team

Status: Draft for Review

Executive Summary

This document outlines the requirements for building a production-grade, distributed rate limiting service using the Token Bucket algorithm. The system will be implemented as a Spring Boot 3 microservice with Redis-backed token storage, providing flexible, scalable API rate limiting capabilities for modern cloud-native applications.

Key Deliverables:

- Spring Boot 3 REST API with configurable rate limiting
- Token Bucket algorithm implementation using Redis for distributed state
- Spring Security 6 integration with JWT authentication
- Annotation-based rate limit configuration with bypass mechanisms
- Comprehensive test suite using Testcontainers
- Docker-based deployment configuration
- Production-ready observability and monitoring

Strategic Value: This project serves as both a reusable infrastructure component and a portfolio demonstration piece, showcasing enterprise-grade backend engineering skills including distributed systems design, security implementation, scalability patterns, and modern DevOps practices.

Problem Statement

Current State

Modern API-driven applications face critical challenges in controlling resource consumption and preventing abuse:

1. **Uncontrolled Resource Consumption:** APIs without rate limiting are vulnerable to resource exhaustion from legitimate traffic spikes or malicious actors
2. **Lack of Fair Usage Policies:** No mechanism to enforce equitable API access across different user tiers or API consumers
3. **Security Vulnerabilities:** Absence of request throttling enables brute force attacks, credential stuffing, and DDoS attempts

4. **Inconsistent Protection:** Microservices environments often implement rate limiting inconsistently or not at all
5. **Poor Observability:** Limited visibility into API usage patterns and abuse attempts

Business Impact

Without proper rate limiting:

- **Service Degradation:** 80-90% of API capacity can be consumed by 5-10% of users during traffic spikes
- **Security Risk:** Brute force attacks can attempt 1000+ login attempts per minute without throttling
- **Cost Overruns:** Uncontrolled API usage leads to unpredictable infrastructure costs
- **Poor User Experience:** Legitimate users suffer degraded performance during abuse incidents
- **Compliance Gaps:** Inability to enforce regulatory requirements for API access controls

Target Opportunity

Build a flexible, production-ready rate limiting solution that:

- Provides microsecond-latency rate limit decisions using Redis
 - Supports multiple rate limiting strategies (per-user, per-IP, per-API key, per-endpoint)
 - Integrates seamlessly with Spring Security authentication flows
 - Scales horizontally without coordination overhead
 - Offers developer-friendly configuration through annotations and properties
-

Goals and Success Metrics

Primary Goals

G1: Deliver Production-Ready Rate Limiting Infrastructure

- Build a reusable rate limiting service that can be deployed in production environments
- Implement industry-standard Token Bucket algorithm with Redis backing
- Achieve <5ms p95 latency overhead for rate limit checks

G2: Demonstrate Enterprise Engineering Maturity

- Implement comprehensive testing strategy (unit, integration, load tests)
- Achieve >85% code coverage with meaningful tests
- Follow security best practices (OWASP, least privilege)
- Provide complete observability (metrics, logs, traces)

G3: Showcase Scalability and Distributed Systems Knowledge

- Support horizontal scaling with stateless application servers
- Handle 10,000+ requests per second with Redis cluster
- Implement graceful degradation when Redis is unavailable
- Demonstrate understanding of distributed systems challenges (race conditions, consistency)

G4: Enable Flexible Configuration

- Support annotation-based rate limit definitions
- Allow runtime configuration updates without deployment
- Provide multiple rate limiting dimensions (user, IP, API key, endpoint)
- Enable bypass mechanisms for administrative access

Success Metrics

Metric	Target	Measurement Method
Rate Limit Decision Latency (p95)	<5ms	Micrometer metrics
Rate Limit Decision Latency (p99)	<10ms	Micrometer metrics
System Throughput	10,000 req/sec	Load testing with JMeter/Gatling
Rate Limit Accuracy	>99.9%	Comparison testing
Test Coverage	>85%	JaCoCo reports
Integration Test Pass Rate	100%	CI/CD pipeline
Redis Connection Pool Efficiency	>90%	Redis metrics
False Rejection Rate	<0.1%	Monitoring in production
API Response Time Overhead	<5% increase	Performance benchmarks
Container Startup Time	<30 seconds	Docker metrics

Key Performance Indicators (KPIs)

Technical KPIs:

- Zero critical security vulnerabilities (SonarQube, Dependabot)
- Mean Time to Recovery (MTTR) <10 minutes
- Uptime availability >99.9%
- Redis cache hit ratio >95%

Portfolio/Resume KPIs:

- GitHub stars/forks as proxy for community validation
 - Code quality metrics (maintainability rating A)
 - Complete technical documentation
 - Production-ready deployment artifacts
-

Target Users

Primary Personas

Persona 1: Backend Software Engineer

- **Profile:** Individual contributor (IC2-IC4) building or enhancing existing APIs
- **Needs:**
 - Drop-in rate limiting solution that requires minimal custom code
 - Clear documentation and code examples
 - Confidence that the implementation follows best practices
- **Success Criteria:** Can integrate rate limiting into existing Spring Boot application in <2 hours

Persona 2: Technical Hiring Manager / Recruiter

- **Profile:** Engineering leader or technical recruiter evaluating candidates
- **Needs:**
 - Evidence of production-ready engineering skills
 - Demonstration of system design thinking
 - Modern tech stack familiarity (Spring Boot 3, Redis, Docker)
- **Success Criteria:** Repository demonstrates senior-level engineering competency

Persona 3: Platform/Infrastructure Engineer

- **Profile:** IC3-IC5 responsible for cross-cutting infrastructure concerns
- **Needs:**
 - Scalable, reliable rate limiting for microservices ecosystem
 - Centralized configuration and monitoring
 - Easy integration with existing auth/security infrastructure
- **Success Criteria:** Can deploy service across multiple environments with standardized configuration

Secondary Personas

Persona 4: Security Engineer

- **Needs:** Robust protection against abuse, visibility into rate limit violations
- **Success Criteria:** Clear security audit trail, configurable violation responses

Persona 5: DevOps/SRE Engineer

- **Needs:** Observable, maintainable service with clear operational runbooks
- **Success Criteria:** Can monitor, troubleshoot, and scale the service confidently

Solution Overview

High-Level Architecture



Core Components

1. Rate Limiting Filter (Spring Security Integration)

- Custom `OncePerRequestFilter` that intercepts all incoming requests
- Extracts rate limiting key (user ID, IP address, API key)
- Queries Redis for current token count
- Applies Token Bucket algorithm logic
- Returns 429 (Too Many Requests) if limit exceeded
- Adds rate limit headers to response

2. Token Bucket Service

- Implements Token Bucket algorithm with configurable capacity and refill rate
- Uses Redis for distributed token storage with atomic operations
- Supports multiple bucket configurations (per-user, per-IP, per-endpoint)
- Handles clock skew and timestamp synchronization

3. Rate Limit Configuration Manager

- Annotation processor for `@RateLimited` and `@RateLimitBypass`
- Externalized configuration via `application.yml`
- Runtime configuration updates via Spring Cloud Config (optional)
- Default rate limits with endpoint-specific overrides

4. Redis Integration Layer

- Lettuce Redis client with connection pooling
- Lua scripts for atomic token bucket operations
- Fallback behavior when Redis is unavailable
- Redis Cluster support for production scalability

5. Monitoring and Observability

- Micrometer metrics for rate limit hits/misses
- Structured logging with correlation IDs
- Health checks for Redis connectivity
- Prometheus-compatible metrics endpoint

Technology Stack

Component	Technology	Version	Justification
Application Framework	Spring Boot	3.2.x	Industry standard, extensive ecosystem
Security	Spring Security	6.2.x	Integrated auth/authz, production-hardened
Cache/Storage	Redis	7.2.x	Microsecond latency, atomic operations
Authentication	JWT	-	Stateless, scalable auth mechanism
Database	PostgreSQL	15.x	User/application data persistence
Testing	JUnit 5 + Testcontainers	5.10.x / 1.19.x	Production-like integration tests
Metrics	Micrometer	1.12.x	Observability standard for Spring
Containerization	Docker + Docker Compose	24.x	Deployment consistency
Build Tool	Maven	3.9.x	Dependency management, reproducible builds
API Documentation	SpringDoc OpenAPI	2.3.x	Interactive API documentation

Token Bucket Algorithm Implementation

Algorithm Overview:

Token Bucket Parameters:

- Capacity (C): Maximum tokens in bucket
- Refill Rate (R): Tokens added per second
- Current Tokens (T): Available tokens at time t

On Request:

1. Calculate tokens to add: $\text{tokens_to_add} = (\text{current_time} - \text{last_refill_time}) \times R$
2. Update current tokens: $T = \min(T + \text{tokens_to_add}, C)$
3. Check if $T \geq \text{tokens_required}$ (usually 1)
4. If yes: $T = T - \text{tokens_required}$, allow request
5. If no: reject request (429)
6. Store T and current_time in Redis

Redis Data Structure:

```
Key: rate_limit:{dimension}:{identifier}:{endpoint}
```

```
Value: {
```

```
  "tokens": <current_token_count>,
```

```
  "last_refill": <timestamp_ms>,
```

```
  "capacity": <max_tokens>,
```

```
  "refill_rate": <tokens_per_second>
```

```
}
```

```
TTL: 2 × (capacity / refill_rate) seconds
```

Lua Script for Atomic Operations: Ensures thread-safe token consumption across distributed instances.

User Journeys

Journey 1: Developer Adds Rate Limiting to New Endpoint

Persona: Backend Software Engineer

Goal: Protect a new API endpoint with rate limiting

Steps:

1. Developer creates new REST endpoint in Spring Boot controller
2. Adds `@RateLimited(requestsPerMinute = 100, burstCapacity = 20)` annotation to method
3. Starts application locally with Docker Compose (includes Redis)
4. Makes test requests using curl/Postman
5. Observes rate limit headers in responses: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`
6. Exceeds rate limit and receives 429 response with `Retry-After` header
7. Commits code and sees integration tests pass in CI pipeline

Success Criteria:

- Endpoint is protected within 5 minutes of adding annotation
- Rate limiting works correctly on first attempt
- Clear error messages if misconfigured

Pain Points Addressed:

- No need to write custom rate limiting logic
- Works with existing Spring Security configuration
- Local development environment matches production

Journey 2: Security Engineer Investigates Rate Limit Violations

Persona: Security Engineer

Goal: Detect and respond to potential API abuse

Steps:

1. Receives alert about elevated rate limit rejections
2. Accesses Grafana dashboard showing rate limit metrics by endpoint, user, and IP
3. Queries structured logs to find specific violating requests
4. Identifies pattern: single IP attempting 5000 login requests in 10 minutes
5. Reviews rate limit configuration for `/api/auth/login` endpoint
6. Updates configuration to reduce rate limit for unauthenticated login attempts
7. Monitors for continued violations
8. Adds IP to blocklist if abuse continues

Success Criteria:

- Can identify violation patterns within 2 minutes
- Configuration changes take effect within 1 minute (Redis cache refresh)
- Clear audit trail of all rate limit decisions

Pain Points Addressed:

- Centralized visibility across all protected endpoints
- Fast response to security incidents
- Flexibility to adjust limits without code deployment

Journey 3: Platform Engineer Deploys to Production

Persona: Platform/Infrastructure Engineer

Goal: Deploy rate limiting service to production Kubernetes cluster

Steps:

1. Reviews project README and deployment documentation
2. Customizes Helm chart values for production environment:
 - Redis Cluster connection string
 - PostgreSQL connection with read replicas
 - Resource limits (CPU, memory)
 - Rate limit defaults for organization
3. Applies configuration via GitOps pipeline (ArgoCD/Flux)
4. Verifies health checks and readiness probes
5. Monitors metrics dashboard for errors
6. Performs load test to validate performance under production traffic

7. Enables rate limiting for 10% of traffic (canary deployment)
8. Gradually rolls out to 100% of traffic

Success Criteria:

- Zero-downtime deployment
- No increase in p95 latency >5%
- Redis connection pool operates efficiently (>90% utilization)

Pain Points Addressed:

- Production-ready configuration examples included
- Health checks prevent routing traffic to unhealthy instances
- Graceful degradation if Redis becomes unavailable

Journey 4: API Consumer Handles Rate Limit Responses

Persona: External API Consumer (Third-party Developer)

Goal: Integrate with rate-limited API without disruption

Steps:

1. Makes API request and receives 200 OK response
2. Checks response headers: `X-RateLimit-Limit: 1000`, `X-RateLimit-Remaining: 999`, `X-RateLimit-Reset: 1643723400`
3. Implements client-side rate limiting based on header values
4. Eventually exceeds rate limit during bulk operation
5. Receives 429 response with `Retry-After: 30` header
6. Implements exponential backoff and retry logic
7. Resumes requests after waiting for `Retry-After` seconds

Success Criteria:

- Clear, standards-compliant rate limit headers
- Predictable rate limit behavior
- Ability to avoid rate limits with proper client implementation

Pain Points Addressed:

- Standard HTTP headers (RFC 6585) for rate limiting
- Retry-After header provides exact wait time
- Consistent behavior across all endpoints

Functional Requirements

Priority Definitions

- **P0 (Must-Have):** Critical for MVP, blocks release
 - **P1 (Should-Have):** Important for production readiness, can be delivered shortly after MVP
 - **P2 (Nice-to-Have):** Enhances functionality, can be deferred to future iterations
-

FR-1: Token Bucket Algorithm Implementation (P0)

Description: Implement distributed Token Bucket rate limiting algorithm using Redis.

Acceptance Criteria:

- ☐ Token bucket maintains capacity, refill rate, and current token count per rate limit key
- ☐ Tokens refill continuously based on elapsed time (not discrete intervals)
- ☐ Algorithm supports configurable capacity (burst) and refill rate (sustained rate)
- ☐ Redis operations are atomic using Lua scripts to prevent race conditions
- ☐ Token bucket state is stored in Redis with appropriate TTL ($2 \times \text{capacity} / \text{refill_rate}$)
- ☐ Algorithm handles clock skew gracefully (timestamp synchronization)
- ☐ Token consumption is precisely 1 token per request (or configurable)
- ☐ System correctly calculates tokens to add: $\min(\text{capacity}, \text{current_tokens} + (\text{time_delta} \times \text{refill_rate}))$
- ☐ Rate limit calculations are accurate within 1% margin of error under load

Technical Details:

- Redis key format: $\text{rate_limit}:\{\text{dimension}\}:\{\text{identifier}\}:\{\text{endpoint_hash}\}$
 - Data structure: Hash with fields $\{\text{tokens}, \text{last_refill_timestamp}, \text{capacity}, \text{refill_rate}\}$
 - Lua script ensures atomic read-calculate-write operations
-

FR-2: Spring Security Filter Integration (P0)

Description: Integrate rate limiting as Spring Security filter in the security chain.

Acceptance Criteria:

- ☐ Custom filter extends `OncePerRequestFilter` to ensure single execution per request
- ☐ Filter is positioned after authentication filters in the security chain
- ☐ Filter executes before reaching controller methods
- ☐ Filter extracts authenticated principal from `SecurityContext` when available
- ☐ Filter gracefully handles unauthenticated requests

- ☐ Filter returns 429 status code when rate limit exceeded
- ☐ Filter allows request to proceed when rate limit not exceeded
- ☐ Filter execution time is <5ms for p95 of requests

Technical Details:

- Filter order: `JwtAuthenticationFilter` → `RateLimitingFilter` → `FilterSecurityInterceptor`
 - Integration point: `SecurityFilterChain` configuration
-

FR-3: Multi-Dimensional Rate Limiting (P0)

Description: Support rate limiting across multiple dimensions (user, IP, API key, endpoint).

Acceptance Criteria:

- ☐ Per-user rate limiting based on authenticated user ID
- ☐ Per-IP rate limiting based on client IP address (with X-Forwarded-For support)
- ☐ Per-API key rate limiting for programmatic access
- ☐ Per-endpoint rate limiting independent of other dimensions
- ☐ Composite rate limiting (e.g., user+endpoint) supported
- ☐ IP extraction handles proxy headers correctly (X-Forwarded-For, X-Real-IP)
- ☐ System supports configuring primary and fallback dimensions
- ☐ Each dimension can have independent rate limit configurations

Technical Details:

- Rate limit key construction: `{dimension_type}:{dimension_value}:{endpoint_pattern}`
 - Supported dimensions: USER, IP, API_KEY, ENDPOINT, COMPOSITE
-

FR-4: Annotation-Based Configuration (P0)

Description: Enable developers to configure rate limits using Java annotations.

Acceptance Criteria:

- ☐ `@RateLimited` annotation applicable to controller methods and classes
- ☐ Annotation supports parameters: `requestsPerMinute`, `requestsPerHour`, `burstCapacity`
- ☐ Annotation supports `dimension` parameter (USER, IP, API_KEY, ENDPOINT)
- ☐ Annotation supports `errorMessage` for custom 429 response messages
- ☐ `@RateLimitBypass` annotation allows exempting specific endpoints
- ☐ Class-level annotations apply to all methods unless overridden
- ☐ Annotation processor validates configuration at startup
- ☐ Invalid configurations cause application startup failure with clear error messages

Example Usage:

```
java

@RestController
@RequestMapping("/api/v1/products")
@RateLimited(requestsPerMinute = 100, burstCapacity = 20)
public class ProductController {

    @GetMapping
    public List<Product> getProducts() { ... }

    @PostMapping
    @RateLimited(requestsPerMinute = 10, burstCapacity = 5, dimension = USER)
    public Product createProduct(@RequestBody Product product) { ... }

    @GetMapping("/admin")
    @RateLimitBypass
    public List<Product> getAdminProducts() { ... }
}
```

FR-5: Rate Limit Response Headers (P0)

Description: Include standard rate limit headers in all API responses.

Acceptance Criteria:

- ☐ `X-RateLimit-Limit` header indicates maximum requests allowed per window
- ☐ `X-RateLimit-Remaining` header indicates remaining requests in current window
- ☐ `X-RateLimit-Reset` header indicates Unix timestamp when limit resets
- ☐ `Retry-After` header included in 429 responses indicating seconds to wait
- ☐ Headers are added to all responses (both successful and rate-limited)
- ☐ Header values are accurate and reflect current rate limit state
- ☐ Headers conform to IETF draft standard for rate limiting

Technical Details:

- Headers added by filter after rate limit check
- Reset time calculated from token bucket refill rate

FR-6: JWT Authentication Integration (P0)

Description: Integrate with JWT-based authentication for user identification.

Acceptance Criteria:

- ☐ System extracts user ID from validated JWT tokens
- ☐ Rate limiting uses authenticated user ID when available
- ☐ System falls back to IP-based rate limiting for unauthenticated requests
- ☐ JWT validation errors do not bypass rate limiting
- ☐ Support for custom JWT claims (e.g., tenant ID, organization ID)
- ☐ Token refresh requests have separate rate limits
- ☐ Expired tokens are handled gracefully

Technical Details:

- JWT parsing using Spring Security's `JwtDecoder`
 - User extraction from `SecurityContext.getAuthentication()`
-

FR-7: Externalized Configuration (P0)

Description: Support runtime configuration via `application.yml` and environment variables.

Acceptance Criteria:

- ☐ Default rate limits configurable via `application.yml`
- ☐ Endpoint-specific overrides supported in configuration
- ☐ Redis connection parameters externalized
- ☐ Environment variable overrides supported for all configuration
- ☐ Configuration reloading without application restart (Spring Cloud Config integration)
- ☐ Configuration validation at startup prevents invalid settings
- ☐ Clear documentation of all configuration properties

Example Configuration:

```
yaml
```

```
rate-limiting:
  enabled: true
default-limits:
  requests-per-minute: 100
  burst-capacity: 20
  dimension: IP
endpoint-overrides:
  - pattern: /api/auth/login
    requests-per-minute: 10
    burst-capacity: 3
  - pattern: /api/users/**
    requests-per-minute: 50
    dimension: USER
redis:
  host: ${REDIS_HOST:localhost}
  port: ${REDIS_PORT:6379}
  password: ${REDIS_PASSWORD:}
```

FR-8: Custom Error Responses (P1)

Description: Provide customizable error responses for rate limit violations.

Acceptance Criteria:

- ☐ 429 response includes JSON body with error details
- ☐ Error response includes error code, message, and timestamp
- ☐ Support for custom error messages per endpoint
- ☐ Error response format matches application's standard error schema
- ☐ Response includes rate limit information (limit, remaining, reset time)
- ☐ Support for internationalization (i18n) of error messages

Example Response:

```
json
```

```
{
  "timestamp": "2026-02-02T10:30:00Z",
  "status": 429,
  "error": "Too Many Requests",
  "message": "Rate limit exceeded. Please try again later.",
  "path": "/api/v1/products",
  "rateLimit": {
    "limit": 100,
    "remaining": 0,
    "reset": 1643723400
  }
}
```

FR-9: Rate Limit Bypass Mechanisms (P1)

Description: Allow administrative and system endpoints to bypass rate limiting.

Acceptance Criteria:

- ☐ `@RateLimitBypass` annotation exempts endpoints from all rate limiting
- ☐ Configuration-based bypass by IP whitelist
- ☐ Configuration-based bypass by API key whitelist
- ☐ Health check and actuator endpoints automatically bypassed
- ☐ Bypass decisions logged for security audit
- ☐ Bypass mechanisms override all rate limit rules

Technical Details:

- Bypass check occurs before rate limit calculation
 - Whitelist stored in application configuration or Redis for runtime updates
-

FR-10: Graceful Degradation (P1)

Description: Handle Redis unavailability without complete service failure.

Acceptance Criteria:

- ☐ Application starts successfully even if Redis is unavailable
- ☐ When Redis is unavailable, system either allows all requests OR denies all requests based on configuration
- ☐ Circuit breaker pattern prevents repeated failed Redis calls
- ☐ Redis reconnection is automatic when service recovers
- ☐ Health check endpoint reflects Redis connection status
- ☐ Degraded mode is clearly indicated in logs and metrics

- ☐ Configuration option for degradation strategy (fail-open vs fail-closed)

Technical Details:

- Resilience4j circuit breaker for Redis operations
 - Fallback configuration: `rate-limiting.fallback-mode: ALLOW | DENY`
-

FR-11: Metrics and Monitoring (P0)

Description: Expose comprehensive metrics for rate limiting operations.

Acceptance Criteria:

- ☐ Counter metric for total rate limit checks
- ☐ Counter metric for rate limit rejections (by endpoint, user, IP)
- ☐ Counter metric for rate limit allows
- ☐ Timer metric for rate limit check latency
- ☐ Gauge metric for current Redis connection pool usage
- ☐ Counter metric for Redis operation failures
- ☐ All metrics tagged with relevant dimensions (endpoint, dimension_type)
- ☐ Metrics exposed via `/actuator/prometheus` endpoint
- ☐ Grafana dashboard template provided

Metric Examples:

- `rate_limit_checks_total{endpoint="/api/products", result="allowed"}`
 - `rate_limit_checks_total{endpoint="/api/products", result="rejected"}`
 - `rate_limit_check_duration_seconds{endpoint="/api/products"}`
-

FR-12: Structured Logging (P1)

Description: Implement comprehensive structured logging for observability.

Acceptance Criteria:

- ☐ All rate limit decisions logged with structured fields
- ☐ Log fields include: timestamp, endpoint, user_id, ip_address, decision, remaining_tokens
- ☐ Rate limit violations logged at WARN level
- ☐ Successful checks logged at DEBUG level
- ☐ Correlation ID included in all log entries
- ☐ Logs use JSON format for machine parsing
- ☐ Sensitive data (API keys, tokens) redacted from logs

Example Log Entry:

json

```
{
  "timestamp": "2026-02-02T10:30:00.123Z",
  "level": "WARN",
  "message": "Rate limit exceeded",
  "correlation_id": "abc-123-def",
  "endpoint": "/api/v1/products",
  "user_id": "user-456",
  "ip_address": "192.168.1.100",
  "decision": "REJECTED",
  "remaining_tokens": 0,
  "limit": 100,
  "reset_time": 1643723400
}
```

FR-13: Health Checks (P0)

Description: Implement comprehensive health checks for the service.

Acceptance Criteria:

- ☐ `/actuator/health` endpoint returns 200 when service is healthy
- ☐ Health check includes Redis connectivity check
- ☐ Health check includes database connectivity check
- ☐ Health check response includes status of each dependency
- ☐ Readiness probe ensures service can handle traffic
- ☐ Liveness probe detects application deadlocks
- ☐ Health checks do not bypass rate limiting (use bypass annotation)

Example Response:

json

```
{
  "status": "UP",
  "components": {
    "redis": {
      "status": "UP",
      "details": {
        "version": "7.2.3",
        "latency_ms": 2
      }
    },
    "db": {
      "status": "UP",
      "details": {
        "database": "PostgreSQL",
        "version": "15.4"
      }
    }
  }
}
```

FR-14: API Documentation (P1)

Description: Provide comprehensive API documentation via OpenAPI/Swagger.

Acceptance Criteria:

- ☐ OpenAPI 3.0 specification generated from code
 - ☐ Interactive API documentation available at `/swagger-ui.html`
 - ☐ Documentation includes rate limit annotations for each endpoint
 - ☐ Example requests and responses included
 - ☐ Authentication requirements documented
 - ☐ Rate limit headers documented in response schemas
-

FR-15: Integration Testing (P0)

Description: Comprehensive integration test suite using Testcontainers.

Acceptance Criteria:

- ☐ Integration tests use real Redis instance via Testcontainers
- ☐ Tests verify rate limiting behavior under concurrent load
- ☐ Tests verify token bucket refill logic over time
- ☐ Tests verify all rate limiting dimensions (user, IP, API key, endpoint)

- ☐ Tests verify graceful degradation when Redis is unavailable
 - ☐ Tests verify rate limit headers in responses
 - ☐ Tests verify 429 error responses
 - ☐ Integration tests run in CI pipeline
 - ☐ Test execution time <2 minutes for full suite
-

FR-16: Concurrent Request Handling (P0)

Description: Correctly handle concurrent requests from the same rate limit key.

Acceptance Criteria:

- ☐ Atomic Redis operations prevent token double-spending
- ☐ System accurately enforces rate limits under high concurrency (1000+ req/sec)
- ☐ Load tests with 10 concurrent threads verify correctness
- ☐ No race conditions allow bypassing rate limits
- ☐ Lua script ensures atomicity for token consumption

Technical Details:

- Use Redis Lua scripts for atomic read-modify-write operations
 - Load testing with JMeter or Gatling to verify
-

FR-17: Database Integration for User Management (P1)

Description: PostgreSQL database for user and application data.

Acceptance Criteria:

- ☐ Users table stores user accounts with unique IDs
 - ☐ API keys table stores programmatic access credentials
 - ☐ Rate limit configuration can reference database entities
 - ☐ Database schema versioning with Flyway/Liquibase
 - ☐ Database connection pooling configured (HikariCP)
 - ☐ Read replicas supported for high availability
-

FR-18: Docker Deployment Configuration (P0)

Description: Complete Docker and Docker Compose configuration for local and production deployment.

Acceptance Criteria:

- ☐ Dockerfile builds minimal production image (<200MB)

- ☐ Multi-stage build separates build and runtime layers
 - ☐ Docker Compose includes Spring Boot app, Redis, and PostgreSQL
 - ☐ Environment variables configure all external dependencies
 - ☐ Container health checks configured
 - ☐ Containers start in correct dependency order
 - ☐ Volume mounts for persistent data (PostgreSQL, Redis AOF)
 - ☐ Docker Compose setup works on first run (`docker-compose up`)
-

FR-19: CI/CD Pipeline (P2)

Description: Automated CI/CD pipeline configuration.

Acceptance Criteria:

- ☐ GitHub Actions or GitLab CI configuration included
 - ☐ Pipeline runs tests on every commit
 - ☐ Pipeline builds Docker image
 - ☐ Pipeline runs security scans (Trivy, Dependabot)
 - ☐ Pipeline publishes code coverage reports
 - ☐ Pipeline tags releases with semantic versioning
-

FR-20: Load Testing Suite (P2)

Description: Performance and load testing suite.

Acceptance Criteria:

- ☐ JMeter or Gatling test plans included
 - ☐ Load tests verify 10,000 req/sec throughput
 - ☐ Load tests verify latency requirements (p95 <5ms)
 - ☐ Load tests verify rate limit accuracy under load
 - ☐ Test reports generated with performance graphs
-

Non-Functional Requirements

NFR-1: Performance

Requirements:

- **Latency:** Rate limit check adds <5ms to p95 request latency, <10ms to p99
- **Throughput:** System handles 10,000+ requests per second per instance
- **Redis Latency:** Redis operations complete in <2ms p95

- **Memory:** Application consumes <512MB heap under normal load
- **CPU:** Application uses <1 CPU core at 10,000 req/sec

Measurement:

- Micrometer metrics for request latency
- Load testing with Gatling or JMeter
- Redis latency monitoring via Redis CLI

Acceptance Criteria:

- ☐ p95 latency <5ms verified by load tests
 - ☐ System sustains 10,000 req/sec for 10 minutes without degradation
 - ☐ Memory usage remains stable (no memory leaks)
-

NFR-2: Scalability

Requirements:

- **Horizontal Scaling:** Stateless application servers scale independently
- **Redis Scaling:** Support Redis Cluster for horizontal scaling
- **Database Scaling:** Support read replicas for user data
- **Linear Scaling:** 2x instances = 2x throughput ($\pm 10\%$)

Acceptance Criteria:

- ☐ Application servers can be added/removed without coordination
 - ☐ Redis Cluster integration tested and documented
 - ☐ No shared state between application instances
 - ☐ Load balancing distributes traffic evenly
-

NFR-3: Availability

Requirements:

- **Target Uptime:** 99.9% availability (43 minutes downtime per month)
- **Zero-Downtime Deployment:** Rolling updates without service interruption
- **Graceful Shutdown:** In-flight requests complete during shutdown
- **Redis Failover:** <30 second failover to Redis replica

Acceptance Criteria:

- ☐ Health checks prevent routing to unhealthy instances

- ☐ Graceful shutdown waits for in-flight requests (max 30 seconds)
 - ☐ Redis Sentinel or Cluster provides automatic failover
-

NFR-4: Security

Requirements:

- **Authentication:** All endpoints except health checks require JWT authentication
- **Authorization:** Rate limit bypass requires administrative privileges
- **Data Protection:** Sensitive data encrypted in transit (TLS) and at rest
- **Security Headers:** HSTS, X-Content-Type-Options, X-Frame-Options included
- **Vulnerability Scanning:** Zero critical/high vulnerabilities in dependencies
- **Audit Logging:** All rate limit decisions logged for security audit

Acceptance Criteria:

- ☐ TLS 1.3 enforced for all external connections
 - ☐ Secrets managed via environment variables or secret management system
 - ☐ Dependabot alerts enabled and addressed within 7 days
 - ☐ OWASP Top 10 vulnerabilities mitigated
-

NFR-5: Observability

Requirements:

- **Metrics:** Prometheus-compatible metrics for all key operations
- **Logging:** Structured JSON logs with correlation IDs
- **Tracing:** Distributed tracing support via OpenTelemetry (optional)
- **Dashboards:** Grafana dashboard for rate limiting metrics
- **Alerting:** Alerts for high error rates, Redis failures, and performance degradation

Acceptance Criteria:

- ☐ Metrics exported via `/actuator/prometheus`
 - ☐ All logs include correlation ID for request tracing
 - ☐ Grafana dashboard template included in repository
 - ☐ Sample alerting rules provided for Prometheus AlertManager
-

NFR-6: Maintainability

Requirements:

- **Code Quality:** SonarQube maintainability rating A
- **Test Coverage:** >85% line coverage, >80% branch coverage
- **Documentation:** README, API docs, architecture diagrams, operational runbooks
- **Code Style:** Consistent formatting enforced by Checkstyle/SpotBugs
- **Dependency Management:** All dependencies <6 months old

Acceptance Criteria:

- ☐ SonarQube analysis passes quality gate
 - ☐ JaCoCo coverage reports meet thresholds
 - ☐ README includes quickstart, configuration, deployment instructions
 - ☐ Architecture diagrams included in `/docs` folder
-

NFR-7: Portability

Requirements:

- **Platform Independence:** Runs on Linux, macOS, Windows (via Docker)
- **Cloud Agnostic:** No vendor-specific dependencies (AWS, GCP, Azure)
- **Container Deployment:** Docker image compatible with Kubernetes, ECS, Cloud Run
- **Configuration Flexibility:** 12-factor app principles for environment-specific configuration

Acceptance Criteria:

- ☐ Docker Compose works on Linux, macOS, Windows
 - ☐ Helm chart provided for Kubernetes deployment
 - ☐ No hardcoded environment-specific values
-

NFR-8: Reliability

Requirements:

- **Error Handling:** All exceptions logged and handled gracefully
- **Circuit Breaker:** Redis operations use circuit breaker pattern
- **Retry Logic:** Transient failures retry with exponential backoff
- **Idempotency:** Rate limit checks are idempotent (safe to retry)
- **Data Consistency:** Token bucket state is eventually consistent

Acceptance Criteria:

- ☐ Chaos testing verifies resilience to Redis failures
 - ☐ Circuit breaker opens after 5 consecutive failures
 - ☐ All errors return appropriate HTTP status codes
-

NFR-9: Compliance

Requirements:

- **Data Residency:** Support for region-specific Redis and database instances
- **Audit Trail:** All rate limit decisions logged with user identification
- **GDPR Compliance:** User data can be exported and deleted
- **Retention Policies:** Logs retained for 90 days, metrics for 30 days

Acceptance Criteria:

- ☐ Configuration supports multiple Redis clusters by region
 - ☐ Audit logs include timestamp, user ID, IP address, decision
 - ☐ User deletion API removes all associated rate limit history
-

NFR-10: Development Experience

Requirements:

- **Local Setup:** Developer can run entire stack locally in <5 minutes
- **Hot Reload:** Code changes reflected without full restart (Spring DevTools)
- **Clear Errors:** Configuration errors provide actionable error messages
- **Examples:** Sample controller with rate limiting annotations included

Acceptance Criteria:

- ☐ `docker-compose up` starts all services successfully
 - ☐ README quickstart guide enables first successful request in <10 minutes
 - ☐ Sample API endpoints demonstrate all rate limiting features
-

Data and Integrations

Data Models

Rate Limit Bucket (Redis)

Key: rate_limit:{dimension}:{identifier}:{endpoint_hash}

Type: Hash

TTL: $2 \times (\text{capacity} / \text{refill_rate})$ seconds

Fields:

```
{
  "tokens": float,          // Current token count (0 to capacity)
  "last_refill": long,      // Unix timestamp (milliseconds) of last refill
  "capacity": int,          // Maximum tokens (burst capacity)
  "refill_rate": float      // Tokens added per second
}
```

Example:

Key: rate_limit:USER:user-123:hash-abc

Value: {

```
  "tokens": 45.7,
  "last_refill": 1643723400123,
  "capacity": 100,
  "refill_rate": 1.67
}
```

User Entity (PostgreSQL)

sql

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(255) NOT NULL UNIQUE,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(50) NOT NULL DEFAULT 'USER',
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_email ON users(email);
```

API Key Entity (PostgreSQL)

sql

```
CREATE TABLE api_keys (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  key_hash VARCHAR(255) NOT NULL UNIQUE,  
  name VARCHAR(255),  
  scopes TEXT[],  
  rate_limit_override JSONB, -- Custom rate limit for this key  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  expires_at TIMESTAMP,  
  last_used_at TIMESTAMP  
);  
  
CREATE INDEX idx_api_keys_user_id ON api_keys(user_id);  
CREATE INDEX idx_api_keys_key_hash ON api_keys(key_hash);
```

Rate Limit Configuration (PostgreSQL - Optional)

```
sql  
  
CREATE TABLE rate_limit_configs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  endpoint_pattern VARCHAR(255) NOT NULL,  
  dimension VARCHAR(50) NOT NULL,  
  requests_per_minute INT NOT NULL,  
  burst_capacity INT NOT NULL,  
  enabled BOOLEAN NOT NULL DEFAULT true,  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_rate_limit_configs_endpoint ON rate_limit_configs(endpoint_pattern);
```

External Integrations

Redis Integration

Purpose: Distributed token storage and atomic rate limit operations

Connection Details:

- **Protocol:** Redis Serialization Protocol (RESP)
- **Client:** Lettuce (async, reactive Redis client)
- **Connection Pool:** Min 5, Max 20 connections
- **Timeout:** 2000ms for operations
- **Retry:** 3 retries with exponential backoff

Operations:

- `EVAL` (Lua script) - Atomic token bucket operations
- `HGET`/`HSET` - Read/write token bucket state
- `EXPIRE` - Set TTL on rate limit keys
- `PING` - Health check

Failure Handling:

- Circuit breaker opens after 5 consecutive failures
- Fallback to configured degradation mode (allow or deny)

PostgreSQL Integration

Purpose: User data, API key management, audit logs

Connection Details:

- **Driver:** PostgreSQL JDBC Driver 42.x
- **Connection Pool:** HikariCP (min 5, max 20 connections)
- **Schema Versioning:** Flyway migrations

Operations:

- User authentication (read-only queries)
 - API key validation (read-only queries)
 - Audit log writes (async, non-blocking)
-

API Design

Rate Limiting Headers (All Responses)

Standard Headers (RFC 6585 compliant):

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 87
X-RateLimit-Reset: 1643723400
```

429 Response Headers:

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1643723400
Retry-After: 30
Content-Type: application/json
```

Sample Endpoints

GET /api/v1/products

Description: Retrieve list of products (rate limited)

Rate Limit: 100 requests/minute, burst 20

Request:

```
http

GET /api/v1/products HTTP/1.1
Host: api.example.com
Authorization: Bearer <jwt_token>
```

Success Response (200 OK):

```
http

HTTP/1.1 200 OK
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 99
X-RateLimit-Reset: 1643723460
Content-Type: application/json

[
  {
    "id": "prod-123",
    "name": "Product A",
    "price": 29.99
  }
]
```

Rate Limit Exceeded Response (429):

```
http
```

HTTP/1.1 429 Too Many Requests

X-RateLimit-Limit: 100

X-RateLimit-Remaining: 0

X-RateLimit-Reset: 1643723460

Retry-After: 30

Content-Type: application/json

```
{
  "timestamp": "2026-02-02T10:30:00Z",
  "status": 429,
  "error": "Too Many Requests",
  "message": "Rate limit exceeded. Please try again in 30 seconds.",
  "path": "/api/v1/products",
  "rateLimit": {
    "limit": 100,
    "remaining": 0,
    "reset": 1643723460
  }
}
```

POST /api/auth/login

Description: User authentication (strict rate limiting)

Rate Limit: 10 requests/minute, burst 3 (per IP address)

Request:

http

POST /api/auth/login HTTP/1.1

Host: api.example.com

Content-Type: application/json

```
{
  "username": "user@example.com",
  "password": "securePassword123"
}
```

Success Response (200 OK):

http

```
HTTP/1.1 200 OK
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 9
X-RateLimit-Reset: 1643723460
```

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

GET /actuator/health

Description: Health check endpoint (bypasses rate limiting)

Request:

```
http
GET /actuator/health HTTP/1.1
Host: api.example.com
```

Response (200 OK):

```
http
HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "UP",
  "components": {
    "redis": {
      "status": "UP",
      "details": {
        "version": "7.2.3",
        "latency_ms": 2
      }
    },
    "db": {
      "status": "UP"
    }
  }
}
```

GET /actuator/prometheus

Description: Prometheus metrics endpoint (bypasses rate limiting)

Request:

```
http
GET /actuator/prometheus HTTP/1.1
Host: api.example.com
```

Response (200 OK):

```
# HELP rate_limit_checks_total Total rate limit checks
# TYPE rate_limit_checks_total counter
rate_limit_checks_total{endpoint="/api/products",result="allowed"} 15234.0
rate_limit_checks_total{endpoint="/api/products",result="rejected"} 342.0

# HELP rate_limit_check_duration_seconds Rate limit check duration
# TYPE rate_limit_check_duration_seconds summary
rate_limit_check_duration_seconds{endpoint="/api/products",quantile="0.95"} 0.004123
rate_limit_check_duration_seconds{endpoint="/api/products",quantile="0.99"} 0.008456
```

Authentication Flow

JWT Token Structure:

```
json
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },
  "payload": {
    "sub": "user-123",
    "email": "user@example.com",
    "role": "USER",
    "iat": 1643720000,
    "exp": 1643723600
  }
}
```

Rate Limit Key Extraction:

- Authenticated requests: `USER:user-123`
 - Unauthenticated requests: `IP:192.168.1.100`
 - API key requests: `API_KEY:key-abc-123`
-

Risks and Mitigations

R-1: Redis Single Point of Failure

Risk Level: HIGH

Impact: Complete rate limiting failure if Redis becomes unavailable

Mitigation Strategies:

- Deploy Redis in master-replica configuration with Redis Sentinel for automatic failover
- Implement circuit breaker pattern to prevent cascading failures
- Configure graceful degradation mode (fail-open or fail-closed based on security requirements)
- Monitor Redis health proactively with alerts

Residual Risk: LOW (with mitigations implemented)

R-2: Clock Skew in Distributed Environment

Risk Level: MEDIUM

Impact: Inaccurate rate limit calculations if server clocks drift

Mitigation Strategies:

- Use Redis server time for all timestamp calculations (TIME command)
- Implement NTP synchronization on all application servers
- Document tolerance for clock skew (± 1 second acceptable)
- Monitor clock drift with automated alerts

Residual Risk: LOW

R-3: Redis Network Latency

Risk Level: MEDIUM

Impact: High latency violates $< 5\text{ms}$ p95 requirement if Redis is geographically distant

Mitigation Strategies:

- Deploy Redis in same datacenter/availability zone as application servers

- Use Redis connection pooling to minimize connection overhead
- Implement Lua scripts to minimize network round-trips
- Monitor Redis latency and alert on degradation

Residual Risk: LOW (with co-location)

R-4: Token Bucket Race Conditions

Risk Level: HIGH

Impact: Users could bypass rate limits under high concurrency without atomic operations

Mitigation Strategies:

- Use Redis Lua scripts for atomic read-modify-write operations
- Comprehensive load testing with concurrent requests (10+ threads)
- Code review focusing on atomicity guarantees
- Integration tests specifically for race conditions

Residual Risk: LOW (with Lua scripts)

R-5: Configuration Errors Allow Bypass

Risk Level: MEDIUM

Impact: Misconfiguration could disable rate limiting or set limits too high

Mitigation Strategies:

- Configuration validation at application startup
- Default to restrictive rate limits (fail-secure)
- Peer review for configuration changes
- Automated testing of configuration scenarios
- Clear documentation of configuration options

Residual Risk: LOW

R-6: Performance Degradation Under Load

Risk Level: MEDIUM

Impact: System cannot meet 10,000 req/sec throughput requirement

Mitigation Strategies:

- Comprehensive load testing before production deployment

- Performance profiling to identify bottlenecks
- Horizontal scaling capability verified under load
- Redis cluster deployment for high-throughput scenarios
- Caching of rate limit configurations

Residual Risk: LOW (with load testing)

R-7: Security Vulnerabilities in Dependencies

Risk Level: MEDIUM

Impact: Exploitable vulnerabilities in Spring Boot, Redis client, or other dependencies

Mitigation Strategies:

- Dependabot automated dependency updates
- Weekly vulnerability scanning with Trivy/Snyk
- Security-first dependency selection
- Rapid patching process for critical vulnerabilities
- Regular security audits

Residual Risk: MEDIUM (ongoing vigilance required)

R-8: Insufficient Testing Coverage

Risk Level: MEDIUM

Impact: Bugs in production due to untested edge cases

Mitigation Strategies:

-  85% code coverage requirement enforced in CI
- Integration tests with Testcontainers for realistic scenarios
- Load testing for performance validation
- Chaos testing for failure scenarios (Redis down, network partitions)
- Manual testing of security-critical flows

Residual Risk: LOW

R-9: Redis Memory Exhaustion

Risk Level: MEDIUM

Impact: Redis runs out of memory, causing evictions or service failure

Mitigation Strategies:

- Configure Redis max memory limit with LRU eviction policy
- Set appropriate TTLs on all rate limit keys ($2 \times \text{capacity} / \text{refill_rate}$)
- Monitor Redis memory usage with alerts at 80% capacity
- Plan for Redis vertical or horizontal scaling
- Periodic cleanup of stale keys

Residual Risk: LOW

R-10: Inadequate Documentation

Risk Level: MEDIUM

Impact: Developers cannot effectively use or maintain the service

Mitigation Strategies:

- Comprehensive README with quickstart guide
- API documentation via OpenAPI/Swagger
- Architecture diagrams and design documentation
- Operational runbooks for common scenarios
- Code comments for complex logic
- Example projects demonstrating usage

Residual Risk: LOW

MVP Scope

In-Scope for MVP (Must-Have)

Core Functionality:

- ☒ Token Bucket algorithm implementation with Redis
- ☒ Spring Security filter integration
- ☒ JWT authentication integration
- ☒ Multi-dimensional rate limiting (user, IP, endpoint)
- ☒ Annotation-based configuration (`@RateLimited`, `@RateLimitBypass`)
- ☒ Rate limit response headers (`X-RateLimit-*`, `Retry-After`)
- ☒ Externalized configuration via `application.yml`
- ☒ 429 error responses with JSON body

Infrastructure:

- ☒ Docker and Docker Compose setup
- ☒ PostgreSQL database with Flyway migrations
- ☒ Redis integration with Lettuce client
- ☒ Health check endpoints

Testing:

- ☒ Unit tests with >85% coverage
- ☒ Integration tests with Testcontainers
- ☒ Load tests validating performance requirements

Observability:

- ☒ Micrometer metrics (Prometheus format)
- ☒ Structured JSON logging
- ☒ Basic Grafana dashboard template

Documentation:

- ☒ README with quickstart guide
- ☒ API documentation (Swagger/OpenAPI)
- ☒ Configuration reference
- ☒ Architecture diagram

Out-of-Scope for MVP (Future Enhancements)

Advanced Features:

- ☒ Distributed rate limiting across multiple Redis clusters
- ☒ Custom rate limit algorithms (Leaky Bucket, Fixed Window, Sliding Window)
- ☒ Rate limit quota management UI
- ☒ Real-time rate limit configuration updates (Spring Cloud Config)
- ☒ Advanced IP geolocation for rate limiting by region
- ☒ ML-based anomaly detection for abuse patterns

Enterprise Features:

- ☒ Multi-tenancy with tenant-specific rate limits
- ☒ Rate limit billing/metering integration
- ☒ Advanced analytics and reporting dashboard

- ❌ SAML/OAuth2 integration (beyond JWT)
- ❌ API gateway integration (Kong, Apigee)

DevOps/Operations:

- ❌ Kubernetes Helm chart (basic provided, advanced features deferred)
- ❌ GitOps deployment with ArgoCD/Flux
- ❌ Automated canary deployments
- ❌ Advanced observability (distributed tracing with Jaeger/Zipkin)
- ❌ Automated chaos testing in CI/CD

Performance Optimization:

- ❌ Local in-memory caching layer (Caffeine) for rate limit configurations
 - ❌ Redis pipelining for batch operations
 - ❌ Adaptive rate limiting based on system load
-

Timeline

Assumptions:

- Single developer, working part-time (10-15 hours/week)
- Prior experience with Spring Boot and Redis
- Access to cloud environment for load testing (AWS free tier or similar)

Phase 1: Foundation (Weeks 1-2)

Week 1: Project Setup and Core Algorithm

- Project structure and dependency setup (Maven/Gradle)
- Redis integration with Lettuce client
- Token Bucket algorithm implementation
- Basic unit tests for algorithm
- Docker Compose setup (Redis + PostgreSQL)

Deliverables:

- Working Token Bucket algorithm with Redis
 - Unit tests with >80% coverage
 - Local development environment
-

Week 2: Spring Security Integration

- Custom rate limiting filter
- Spring Security filter chain integration
- JWT authentication setup
- User entity and repository
- Integration tests with Testcontainers

Deliverables:

- Rate limiting filter integrated with Spring Security
 - JWT authentication flow working
 - Integration tests passing
-

Phase 2: Configuration and Error Handling (Weeks 3-4)

Week 3: Annotation-Based Configuration

- `@RateLimited` annotation implementation
- `@RateLimitBypass` annotation
- Annotation processor for extracting configuration
- Externalized configuration via `application.yml`
- Endpoint-specific rate limit overrides

Deliverables:

- Annotation-based configuration working
 - Configuration validation at startup
 - Example controller with rate limiting
-

Week 4: Error Handling and Headers

- Rate limit response headers implementation
- 429 error response with JSON body
- Custom error messages
- Graceful degradation when Redis unavailable
- Circuit breaker implementation

Deliverables:

- Complete error handling
 - Rate limit headers in all responses
 - Resilience to Redis failures
-

Phase 3: Observability and Testing (Weeks 5-6)

Week 5: Metrics and Logging

- Micrometer metrics integration
- Structured logging with correlation IDs
- Prometheus endpoint configuration
- Grafana dashboard template
- Health check endpoints

Deliverables:

- Comprehensive metrics exposed
 - Structured logging implemented
 - Grafana dashboard for monitoring
-

Week 6: Comprehensive Testing

- Expand unit test coverage to >85%
- Integration tests for all rate limiting scenarios
- Load tests with Gatling/JMeter (10,000 req/sec)
- Performance profiling and optimization
- Chaos testing (Redis failures)

Deliverables:

- Test coverage >85%
 - Load test results documenting performance
 - Performance optimizations applied
-

Phase 4: Documentation and Polish (Week 7)

Week 7: Documentation and Deployment

- Comprehensive README with quickstart
- API documentation (Swagger/OpenAPI)
- Architecture diagrams
- Configuration reference
- Deployment guide (Docker, Kubernetes)
- Code cleanup and refactoring

Deliverables:

- Production-ready documentation
 - Clean, maintainable codebase
 - Deployment artifacts (Dockerfile, Helm chart)
-

Phase 5: Optional Enhancements (Week 8+)

Post-MVP Enhancements (if time permits):

- CI/CD pipeline (GitHub Actions)
 - Additional rate limiting dimensions (API key)
 - Advanced configuration (rate limit per user tier)
 - Performance dashboard with real-time metrics
 - Video walkthrough/demo
-

Milestone Summary

Milestone	Target Date	Success Criteria
M1: Core Algorithm	End of Week 2	Token Bucket working with Redis, JWT auth integrated
M2: Configuration	End of Week 4	Annotation-based config, error handling complete
M3: Observability	End of Week 6	Metrics, logging, comprehensive testing complete
M4: MVP Launch	End of Week 7	Production-ready with documentation, deployable

Total Timeline: 7 weeks (10-15 hours/week) = 70-105 hours

Out of Scope

The following items are explicitly **not** included in this project:

Functional Exclusions

1. User Interface/Frontend:

- No web UI for rate limit configuration
- No admin dashboard for viewing rate limit statistics
- No user-facing portal for quota management

2. Alternative Rate Limiting Algorithms:

- Leaky Bucket algorithm
- Fixed Window algorithm
- Sliding Window Log algorithm
- Sliding Window Counter algorithm

3. Advanced Authentication:

- OAuth2 server implementation
- SAML integration
- Multi-factor authentication (MFA)
- Social login providers (Google, GitHub, etc.)

4. API Gateway Integration:

- Kong plugin development
- Apigee integration
- AWS API Gateway custom authorizer
- NGINX rate limiting integration

5. Billing/Metering:

- Usage-based billing integration
- Quota purchase/upgrade flows
- Payment processing
- Invoice generation

6. Multi-Tenancy:

- Tenant isolation
- Per-tenant rate limit configurations
- Tenant-specific Redis instances

7. Advanced Analytics:

- Machine learning-based abuse detection

- Predictive rate limiting
- User behavior analytics
- Custom reporting engine

Technical Exclusions

1. Caching Layers:

- Local in-memory cache (Caffeine, Guava)
- CDN integration
- Varnish or edge caching

2. Alternative Data Stores:

- Cassandra for rate limit storage
- DynamoDB integration
- MongoDB support
- In-memory grid (Hazelcast, Apache Ignite)

3. Advanced Observability:

- Distributed tracing (Jaeger, Zipkin, OpenTelemetry)
- Custom APM integration (New Relic, Datadog)
- Advanced anomaly detection
- Predictive alerting

4. Enterprise Integration:

- LDAP/Active Directory integration
- Enterprise service bus (ESB) integration
- SIEM integration
- DLP (Data Loss Prevention) integration

5. Mobile/Desktop SDKs:

- iOS SDK for rate limiting awareness
- Android SDK
- Desktop client SDK

Operational Exclusions

1. Advanced Deployment:

- Blue-green deployment automation
- Canary deployment with traffic splitting
- Multi-region active-active deployment

- Disaster recovery automation

2. SLA/SLO Management:

- Formal SLA definitions and enforcement
- SLO tracking and reporting
- Error budget calculations

3. Compliance Certifications:

- SOC 2 compliance
- HIPAA compliance
- PCI-DSS compliance
- ISO 27001 certification

4. 24/7 Support:

- On-call rotation
 - Incident management process
 - Customer support portal
-

Open Questions

Technical Questions

Q1: Should we support rate limiting based on request payload size?

- **Context:** Large payloads consume more resources than small ones
 - **Options:**
 - (A) Use token cost proportional to payload size
 - (B) Separate rate limit for upload/download bandwidth
 - (C) Defer to future iteration
 - **Stakeholders:** Engineering, Product
 - **Decision Needed By:** Week 2
 - **Current Recommendation:** Option C (defer) - adds complexity, not critical for MVP
-

Q2: What should be the default fallback behavior when Redis is unavailable?

- **Context:** Fail-open (allow all) vs fail-closed (deny all)
- **Options:**
 - (A) Fail-open: Allow all requests (security risk, availability priority)
 - (B) Fail-closed: Deny all requests (availability risk, security priority)

- (C) Configurable per environment
 - **Stakeholders:** Security, Engineering, Operations
 - **Decision Needed By:** Week 4
 - **Current Recommendation:** Option C with default fail-closed for production
-

Q3: Should rate limits apply to internal service-to-service calls?

- **Context:** Microservices may call each other's APIs
 - **Options:**
 - (A) Exempt all internal service calls (requires service authentication)
 - (B) Apply rate limits uniformly
 - (C) Separate rate limit tier for internal calls
 - **Stakeholders:** Architecture, Engineering
 - **Decision Needed By:** Week 3
 - **Current Recommendation:** Option A with service-to-service JWT claims
-

Q4: How should we handle rate limit synchronization across multiple Redis instances?

- **Context:** Redis Cluster or multiple independent Redis instances
 - **Options:**
 - (A) Single Redis Cluster with hash-slot based sharding
 - (B) Multiple independent Redis instances (no synchronization)
 - (C) Redis Sentinel for high availability, single master
 - **Stakeholders:** Infrastructure, Engineering
 - **Decision Needed By:** Week 5 (for load testing)
 - **Current Recommendation:** Option C for MVP, Option A for production scale
-

Product Questions

Q5: Should different user tiers (free, pro, enterprise) have different default rate limits?

- **Context:** Common SaaS pattern to enforce pricing tiers via rate limits
- **Options:**
 - (A) Yes, implement user-tier based rate limiting
 - (B) No, uniform rate limits for all users (simplify MVP)

- (C) Support via configuration but don't implement tier logic
 - **Stakeholders:** Product, Engineering
 - **Decision Needed By:** Week 3
 - **Current Recommendation:** Option C - support in config, leave tier logic to application
-

Q6: Should we provide a mechanism for users to request rate limit increases?

- **Context:** Users may hit limits during legitimate usage spikes
 - **Options:**
 - (A) Manual request process (email, support ticket)
 - (B) Self-service UI for rate limit increase requests
 - (C) Automatic temporary burst allowance
 - **Stakeholders:** Product, Customer Success
 - **Decision Needed By:** Post-MVP
 - **Current Recommendation:** Option A for MVP, Option B for future
-

Operational Questions

Q7: What are the Redis hosting requirements for production?

- **Context:** Need to size Redis instance appropriately
 - **Options:**
 - (A) Self-hosted Redis on EC2/Compute Engine
 - (B) Managed Redis (ElastiCache, MemoryStore, Azure Cache)
 - (C) Redis Cloud (Redis Enterprise)
 - **Stakeholders:** Infrastructure, Finance
 - **Decision Needed By:** Before production deployment
 - **Current Recommendation:** Option B for simplicity and reliability
-

Q8: Should we implement automated alerting for rate limit abuse patterns?

- **Context:** Proactive detection of abuse vs reactive investigation
- **Options:**
 - (A) Automated alerts via PagerDuty/Opsgenie
 - (B) Daily/weekly summary reports

- (C) Manual monitoring via dashboards
 - **Stakeholders:** Security, Operations
 - **Decision Needed By:** Week 6
 - **Current Recommendation:** Option A for critical endpoints (login, payment)
-

Q9: How should we handle rate limit migration/evolution (changing algorithms or limits)?

- **Context:** Need to update rate limits without disrupting users
 - **Options:**
 - (A) Gradual rollout with feature flags
 - (B) Blue-green deployment of new limits
 - (C) Immediate cutover during maintenance window
 - **Stakeholders:** Engineering, Operations
 - **Decision Needed By:** Post-MVP
 - **Current Recommendation:** Option A for flexibility
-

Q10: Should we support custom rate limit windows (e.g., per day, per week)?

- **Context:** Some use cases benefit from longer windows (daily API quotas)
 - **Options:**
 - (A) Support arbitrary time windows in Token Bucket configuration
 - (B) Only support per-second/per-minute via refill rate
 - (C) Implement separate Fixed Window algorithm for daily quotas
 - **Stakeholders:** Product, Engineering
 - **Decision Needed By:** Week 2
 - **Current Recommendation:** Option B for MVP - Token Bucket can simulate longer windows via low refill rates
-

Assumptions

Technical Assumptions

1. **Technology Stack:**
 - Spring Boot 3.2+ and Spring Security 6.2+ are acceptable framework choices
 - Redis 7.2+ is available and supported in target deployment environment

- PostgreSQL 15+ is the relational database of choice
- Docker and Docker Compose are available for local development
- Java 17+ is the target JDK

2. Infrastructure:

- Redis latency to application servers is <2ms (same datacenter/AZ)
- Network is reliable with <0.1% packet loss
- NTP synchronization is available on all servers
- Kubernetes or Docker-based deployment environment
- Load balancer supports sticky sessions if needed

3. Scale:

- Single application instance can handle 10,000 requests/second
- Redis can handle 100,000+ operations/second (standard for Redis)
- Most rate limit decisions complete in <5ms
- Concurrent users per instance: ~1,000
- Total unique rate limit keys: <1 million per instance

4. Security:

- JWT tokens are signed with HS256 or RS256
- TLS/SSL is terminated at load balancer or API gateway
- Secrets are managed via environment variables or secret management system
- Application runs in containerized environment with network isolation

Product Assumptions

1. User Behavior:

- Legitimate users will respect rate limit headers and implement client-side throttling
- Rate limit violations are primarily accidental (traffic spikes) not malicious
- Users understand HTTP 429 status code and Retry-After semantics

2. Use Cases:

- Primary use case is protecting REST APIs in microservices architecture
- Rate limiting is applied uniformly across all environments (dev, staging, prod)
- Most endpoints need simple rate limiting (requests per minute)

3. Configuration:

- Rate limits are relatively stable (changed monthly, not hourly)
- Default rate limits are acceptable for 80% of endpoints
- Endpoint-specific overrides are needed for <20% of endpoints

Operational Assumptions

1. Deployment:

- Application is deployed using CI/CD pipeline (GitHub Actions, GitLab CI, Jenkins)
- Rolling updates with health checks are supported by orchestration platform
- Zero-downtime deployment is a requirement
- Canary deployments are nice-to-have but not required

2. Monitoring:

- Prometheus is available for metrics scraping
- Grafana is available for dashboard visualization
- ELK stack or similar is available for log aggregation
- Alert management system (PagerDuty, Opsgenie) is available

3. Data Retention:

- Rate limit state in Redis is ephemeral (no backup required)
- Audit logs are retained for 90 days
- Metrics are retained for 30 days
- Database backups are automated and tested

Development Assumptions

1. Team:

- Developer has prior experience with Spring Boot and Redis
- Developer has access to Docker and local development environment
- Developer can allocate 10-15 hours per week for 7 weeks
- Code review and feedback are available from peers or mentors

2. Testing:

- Load testing can be performed in cloud environment (AWS free tier or similar)
- Integration tests can run in CI pipeline (<5 minutes total)
- Test data generation tools are available (Faker, JUnit @DataProvider)

3. Documentation:

- Documentation is written in Markdown
- Diagrams are created using Mermaid, PlantUML, or draw.io
- API documentation is auto-generated from code (Swagger/OpenAPI)

Success Criteria

The MVP will be considered successful when:

Technical Success Criteria

1. Functionality:

- ☒ All P0 functional requirements implemented and tested
- ☒ Integration tests pass at >99% success rate
- ☒ Load tests demonstrate 10,000 req/sec throughput
- ☒ Rate limiting accuracy >99.9% under concurrent load

2. Performance:

- ☒ p95 latency <5ms for rate limit checks
- ☒ p99 latency <10ms for rate limit checks
- ☒ Redis operations complete in <2ms p95
- ☒ Application memory usage <512MB under load

3. Reliability:

- ☒ System handles Redis failures gracefully (circuit breaker works)
- ☒ Health checks accurately reflect system status
- ☒ Zero race conditions detected in load testing
- ☒ No critical security vulnerabilities in code or dependencies

4. Testing:

- ☒ Unit test coverage >85% (line coverage)
- ☒ Integration test coverage for all major flows
- ☒ Load test results documented and meet requirements
- ☒ Chaos testing validates resilience

5. Observability:

- ☒ Prometheus metrics exported for all key operations
- ☒ Structured logs include correlation IDs
- ☒ Grafana dashboard displays real-time rate limiting metrics
- ☒ Alert rules defined for critical failures

Documentation Success Criteria

1. README:

- ☒ Quickstart guide enables first successful request in <10 minutes
- ☒ Configuration reference documents all options
- ☒ Architecture diagram explains system design

- ☒ Deployment guide covers Docker and Kubernetes

2. Code Quality:

- ☒ SonarQube maintainability rating A
- ☒ No critical or high-severity code smells
- ☒ Consistent code formatting (Checkstyle/SpotBugs)
- ☒ Meaningful comments for complex logic

3. API Documentation:

- ☒ OpenAPI 3.0 specification auto-generated
- ☒ Swagger UI accessible and functional
- ☒ All endpoints documented with examples

Portfolio/Resume Success Criteria

1. Demonstration Value:

- ☒ GitHub repository is public with comprehensive README
- ☒ Code demonstrates production-grade engineering practices
- ☒ Technology stack is modern and relevant (Spring Boot 3, Redis 7)
- ☒ Project showcases system design thinking

2. Completeness:

- ☒ Docker Compose setup works on first run
- ☒ CI/CD pipeline (optional) demonstrates DevOps skills
- ☒ No placeholder TODOs or incomplete features in MVP
- ☒ Project is ready to deploy to cloud environment

3. Community Validation:

- ☒ Code quality badges (coverage, build status) displayed in README
- ☒ GitHub stars/forks indicate community interest (optional, post-launch)
- ☒ Blog post or technical writeup explains design decisions (optional)

Appendix

Glossary

- **Token Bucket:** Rate limiting algorithm that allows bursts while maintaining average rate
- **Burst Capacity:** Maximum number of tokens (requests) allowed in a short period
- **Refill Rate:** Rate at which tokens are added to the bucket (requests per second)
- **JWT (JSON Web Token):** Compact, URL-safe token format for authentication

- **Redis:** In-memory data structure store used as cache and message broker
- **Lua Script:** Scripting language for atomic operations in Redis
- **Circuit Breaker:** Design pattern that prevents cascading failures
- **Testcontainers:** Java library for integration testing with Docker containers
- **Micrometer:** Application metrics facade (similar to SLF4J for logging)

References

- **IETF RFC 6585:** HTTP Status Code 429 (Too Many Requests)
- **Token Bucket Algorithm:** https://en.wikipedia.org/wiki/Token_bucket
- **Spring Boot Documentation:** <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- **Spring Security Documentation:** <https://docs.spring.io/spring-security/reference/>
- **Redis Documentation:** <https://redis.io/documentation>
- **Testcontainers Documentation:** <https://testcontainers.com/>
- **12-Factor App Methodology:** <https://12factor.net/>

Revision History

Version	Date	Author	Changes
1.0	2026-02-02	Engineering Team	Initial draft for review

Document Status: Draft for Review
Next Review Date: 2026-02-09
Approvers: Engineering Lead, Product Manager, Security Lead